Tabla de contenido

Sección 01 · Arquitectura de Comunicación 2
Sección 02 · Autenticación & Usuarios 3
2.1 POST /api/v1/auth/login 3
2.2 POST /api/v1/auth/refresh 4
2.3 UsuarioDto — Modelo de usuario 4
Sección 03 · Productos, Servicios & Clientes 5
3.1 GET /api/v1/productos 5
3.2 GET /api/v1/productos/{id} 5
3.3 GET /api/v1/clientes/{id} 6
Sección 04 · Ventas & Operaciones de Caja 6
4.1 POST /api/v1/ventas — Contrato principal 6
4.2 POST /api/v1/caja/inicio-dia 7
4.3 POST /api/v1/caja/movimiento 8
4.4 POST /api/v1/caja/cierre-dia 8
Sección 05 · Órdenes Web & Pagos 9
5.1 POST /api/v1/ordenes 9
5.2 GET /api/v1/ordenes/{id}/estado 9
5.3 POST /api/v1/pagos/simular 10
5.4 GET /api/v1/cuentascobrar/{clienteId} 10
5.5 GET /health 11
Sección 06 · Eventos & Comandos NServiceBus 11
6.1 VentaRealizadaOfflineMessage [CMD] 11
6.2 VentaSincronizadaEvent [EVT] 11
6.3 InventarioActualizadoEvent [EVT] 12
6.4 AplicarTransaccionesOfflineCommand [CMD] 12
6.5 OrdenCambioEstadoEvent [EVT] 12
6.6 ReconciliacionCompletadaEvent [EVT] 13
Sección 07 · Modelos Internos del Core (P2) 13
7.1 FacturaDto — Documento fiscal completo 13
7.2 CotizacionDto — Flujo a factura 14
7.3 AuditLogEntry — Tabla inmutable de auditoría 14
7.4 IntegrationLogEntry — Log de Integración 15
Sección 08 · Tabla Resumen de Endpoints REST 15
Sección 09 · Tabla Resumen de Mensajes NServiceBus 16

Documentación oficial para la creación y Consumo APIs

Sección 01 · Arquitectura de Comunicación

El sistema opera en 4 capas desacopladas. Ningún canal (Caja, Website) accede directamente al Core; toda comunicación pasa por Integración.

Capa	*Persona*	*Stack*
Caja POS	P1	WinForms - LocalDB - NServiceBus Local
Core Backend	P2	ASP.NET - SQL Server - EF Core - Identity
Integración	P3	ASP.NET Core - NServiceBus - SQL Express Mirror
Website	P4	Blazor / Razor Pages - ASP.NET Core

Regla cardinal: El Core es la única fuente de verdad. Todo inventario en Integración y Caja es caché con TTL. Las transacciones offline son eventuales, nunca definitivas hasta confirmación del Core.

Sección 02 · Autenticación & Usuarios

2.1 POST /api/v1/auth/login

POST `/api/v1/auth/login` *Autenticar usuario, retorna JWT + refresh token*

Propietario: P3 (proxy P2) **Consumidor: P1 Caja** **Consumidor: P4 Website** **CRÍTICO**

Request Body

```
public record LoginRequest
{
    [Required]
    public string Username { get; init; }

    [Required]
    public string Password  { get; init; }

    // "Cajero" | "Admin" | "Cliente"
    public string?  Rol     { get; init; }
}
```

Response 200

```
public record LoginResponse
{
    public string  AccessToken  { get; init; }
    public string  RefreshToken { get; init; }
    public int     ExpiresIn    { get; init; }  // 3600 segundos
    public string  Rol          { get; init; }
    public string  UserId       { get; init; }
    public string  Nombre       { get; init; }
    public string? SucursalId   { get; init; }
}
```

Errores

```
401 → { "error": "Credenciales inválidas" }
423 → { "error": "Cuenta bloqueada", "unlockAt": "2025-07-01T12:00:00Z" }
```

2.2 POST /api/v1/auth/refresh

POST `/api/v1/auth/refresh` *Renovar token sin re-login*

Propietario: P3 **Consumidor: P1** **Consumidor: P4**

Request Body

```
public record RefreshRequest
{
    [Required]
    public string RefreshToken { get; init; }
}
```

Response 200

```
public record RefreshResponse
{
    public string AccessToken  { get; init; }
    public string RefreshToken { get; init; }
    public int    ExpiresIn    { get; init; }
}
```

Errores

```
401 → { "error": "Refresh token inválido o expirado" }
```

2.3 UsuarioDto — Modelo de usuario

GET `/api/v1/usuarios/{id}` *Detalle de usuario con rol*

Propietario: P2 Core **Consumidor: P3**

```
public record UsuarioDto
{
    public Guid    Id                { get; init; }
    public string  UserName          { get; init; }
    public string  NombreCompleto    { get; init; }
    public string  Email             { get; init; }
    // "Admin" | "Mantenimiento" | "Consulta" | "Cajero"
    public string  Rol               { get; init; }
    public string? SucursalId        { get; init; }
    public bool    Activo            { get; init; }
    public DateTimeOffset CreadoEn   { get; init; }
    public DateTimeOffset? UltimoLogin { get; init; }
}
```

Sección 03 · Productos, Servicios & Clientes

3.1 GET /api/v1/productos

GET `/api/v1/productos` *Lista paginada con filtros — sirve desde mirror en modo offline*

Propietario: P3 **Consumidor: P1 Caja** **Consumidor: P4 Website**

Query Params (todos opcionales)

```
?page=1  &pageSize=20  &nombre=radiador  &codigo=RAD-001
&categoriaId=3  &soloActivos=true  &conStock=true
```

Response 200

```
public record ProductoPagedResult
{
    public IReadOnlyList<ProductoResumenDto> Items      { get; init; }
    public int     Total      { get; init; }
    public int     Page       { get; init; }
    public int     PageSize   { get; init; }
    public bool    FromMirror { get; init; }  // true = caché local
}

public record ProductoResumenDto
{
    public int     Id         { get; init; }
    public string  Codigo     { get; init; }
    public string  Nombre     { get; init; }
    public decimal Precio     { get; init; }
    public int     Stock      { get; init; }
    public string  Categoria  { get; init; }
    public bool    EsActivo   { get; init; }
}
```

3.2 GET /api/v1/productos/{id}

GET `/api/v1/productos/{id}` *Detalle completo con stock disponible*

Propietario: P3 **Consumidor: P1** **Consumidor: P4** **CRITICO**

```
public record ProductoDetalleDto
{
    public int     Id             { get; init; }
    public string  Codigo         { get; init; }
    public string  Nombre         { get; init; }
    public string? Descripcion    { get; init; }
    public decimal Precio         { get; init; }
    public decimal? PrecioOferta  { get; init; }
    public int     Stock          { get; init; }
    public int     StockMinimo    { get; init; }
    public int     CategoriaId    { get; init; }
    public string  Categoria      { get; init; }
    public bool    EsActivo       { get; init; }
    public DateTimeOffset UltimaSync { get; init; }
}
```

Errores

```
// 404 → { "error": "Producto no encontrado" }
```

3.3 GET /api/v1/clientes/{id}

GET `/api/v1/clientes/{id}` *Datos del cliente — retorna Cliente Anónimo si no existe*

Propietario: P3 **Consumidor: P1**

```
public record ClienteDto
{
    public Guid    Id             { get; init; }
    public bool    EsAnonimo      { get; init; }  // GUID fijo en seed
    public string  Nombre         { get; init; }
    public string? RncCedula      { get; init; }
    public string? Telefono       { get; init; }
    public string? Email          { get; init; }
    public decimal LimiteCredito  { get; init; }
    public decimal SaldoPendiente { get; init; }
}
```

Sección 04 · Ventas & Operaciones de Caja

4.1 POST /api/v1/ventas — Contrato principal

POST `/api/v1/ventas` *Crear venta/factura. Requiere header Idempotency-Key*

Propietario: P3 → P2 **Consumidor: P1 Caja** **CRITICO**

Headers requeridos

```
Idempotency-Key: {Guid}
Authorization: Bearer {jwt}
```

Request Body

```
public record CrearVentaRequest
{
    public Guid    ClienteId       { get; init; }
    public string  CajeroId        { get; init; }
    public string  SucursalId      { get; init; }
    // "Efectivo" | "Tarjeta" | "Transferencia"
    public string  MetodoPago      { get; init; }
    public decimal MontoRecibido   { get; init; }
    public IReadOnlyList<LineaVentaDto> Lineas { get; init; }
}

public record LineaVentaDto
{
    public int     ProductoId      { get; init; }
    public int     Cantidad        { get; init; }
    public decimal PrecioUnitario  { get; init; }
    public decimal? Descuento      { get; init; }
}
```

Response 201

```
public record VentaResponse
{
    public Guid    FacturaId        { get; init; }
    public string  NumeroFactura    { get; init; }
    public decimal Subtotal         { get; init; }
    public decimal Itbis            { get; init; }
    public decimal Total            { get; init; }
```

```
    public decimal Cambio           { get; init; }
    public DateTimeOffset FechaHora { get; init; }
    // "Confirmada" | "Pendiente"
    public string  Estado           { get; init; }
    public bool    DesdeOffline     { get; init; }
}
```

Errores

```
409 → Idempotency-Key ya procesada (retorna respuesta original)
422 → Stock insuficiente / precio modificado / cliente inválido
503 → Core no disponible (solo si offline bloqueado)
```

4.2 POST /api/v1/caja/inicio-dia

POST `/api/v1/caja/inicio-dia` *Apertura de caja con monto inicial*

Propietario: P3 **Consumidor: P1** **CRITICO**

Request

```
public record InicioDiaRequest
{
    public string  SucursalId   { get; init; }
    public string  CajeroId     { get; init; }
    public decimal MontoInicial { get; init; }
    public DateTimeOffset Fecha { get; init; }
}
```

Response 200

```
public record InicioDiaResponse
{
    public Guid   SesionCajaId  { get; init; }
    public string Estado        { get; init; }
    public DateTimeOffset Inicio{ get; init; }
}
```

4.3 POST /api/v1/caja/movimiento

POST `/api/v1/caja/movimiento` *Entrada o salida de efectivo con motivo y firma digital*

Propietario: P3 **Consumidor: P1**

Request

```
public record MovimientoCajaRequest
{
    public Guid    SesionCajaId { get; init; }
    public string  Tipo         { get; init; }  // "IN"|"OUT"
    public decimal Monto        { get; init; }
    public string  Motivo       { get; init; }
    public string  FirmaDigital { get; init; }
}
```

Response 200

```
public record MovimientoCajaResponse
{
    public Guid    MovimientoId { get; init; }
    public decimal SaldoActual  { get; init; }
    public DateTimeOffset FechaHora { get; init; }
}
```

4.4 POST /api/v1/caja/cierre-dia

POST `/api/v1/caja/cierre-dia` *Cuadre de caja y cierre de sesión*

Propietario: P3 **Consumidor: P1**

Request

```
public record CierreDiaRequest
{
    public Guid    SesionCajaId             { get; init; }
    public decimal MontoContadoEfectivo     { get; init; }
}
```

Response 200

```
public record CierreDiaResponse
{
    public decimal MontoSistema    { get; init; }
    public decimal MontoContado    { get; init; }
    public decimal Diferencia      { get; init; }
    public bool    CuadreCorrecto  { get; init; }
    public IReadOnlyList<ResumenMetodoPagoDto> PorMetodo { get; init; }
}

public record ResumenMetodoPagoDto
{
    public string  MetodoPago  { get; init; }
    public decimal Total       { get; init; }
    public int     Cantidad    { get; init; }
}
```

Sección 05 · Órdenes Web & Pagos

5.1 POST /api/v1/ordenes

POST `/api/v1/ordenes` *Crear orden desde portal web — flujo asíncrono*

Propietario: P3 → P2 **Consumidor: P4 Website** **CRITICO**

Request

```
public record CrearOrdenRequest
{
    public Guid    ClienteId   { get; init; }
    public string? Notas       { get; init; }
    public DireccionEntregaDto?
                   Entrega     { get; init; }
    public IReadOnlyList<LineaOrdenDto> Lineas { get; init; }
}
public record LineaOrdenDto
{
    public int  ProductoId  { get; init; }
    public int  Cantidad    { get; init; }
}
```

Response 202 (Async)

```
public record OrdenCreadaResponse
{
    public Guid   OrdenId    { get; init; }
    // "Pendiente"
    public string Estado     { get; init; }
    public decimal Total     { get; init; }
    public string  PollUrl   { get; init; }
}
```

Errores

```
// 422 → Stock insuficiente
// 401 → No autenticado
```

5.2 GET /api/v1/ordenes/{id}/estado

GET `/api/v1/ordenes/{id}/estado` *Seguimiento de orden por el cliente*

Propietario: P3 **Consumidor: P4**

```
public record EstadoOrdenDto
{
    public Guid   OrdenId       { get; init; }
    // "Pendiente"|"Procesando"|"Listo"|"Entregado"|"Cancelado"
    public string Estado        { get; init; }
    public DateTimeOffset CreadaEn        { get; init; }
    public DateTimeOffset? ActualizadaEn  { get; init; }
    public IReadOnlyList<HistorialEstadoDto> Historial { get; init; }
}

public record HistorialEstadoDto
{
    public string Estado        { get; init; }
    public DateTimeOffset Fecha { get; init; }
    public string? Nota         { get; init; }
}
```

5.3 POST /api/v1/pagos/simular

POST `/api/v1/pagos/simular` *Simular pago con procesador externo vía Integración*

Propietario: P3 (ext. processor) **Consumidor: P4**

Request

```
public record SimularPagoRequest
{
    public Guid    OrdenId    { get; init; }
    public decimal Monto      { get; init; }
    // "Tarjeta" | "PayPal"
    public string  MetodoPago { get; init; }
    public string? TokenPago  { get; init; }
}
```

Response 200

```
public record SimularPagoResponse
{
    public Guid   TransaccionId { get; init; }
    // "Aprobado" | "Rechazado"
    public string Resultado     { get; init; }
    public string? CodigoAuth   { get; init; }
    public string? MensajeError { get; init; }
}
```

5.4 GET /api/v1/cuentascobrar/{clienteId}

GET `/api/v1/cuentascobrar/{clienteId}` *CxC pendientes del cliente con abonos*

Propietario: P3 → P2 **Consumidor: P1**

```
public record CuentaPorCobrarDto
{
    public Guid    Id                 { get; init; }
    public Guid    FacturaId          { get; init; }
    public string  NumeroFactura      { get; init; }
    public decimal MontoOriginal      { get; init; }
    public decimal MontoPendiente     { get; init; }
    public DateTimeOffset FechaVencimiento { get; init; }
    // "Pendiente" | "Parcial" | "Pagada" | "Vencida"
    public string  Estado             { get; init; }
    public IReadOnlyList<AbonoDto> Abonos { get; init; }
}
public record AbonoDto
{
    public decimal Monto         { get; init; }
    public string  MetodoPago    { get; init; }
    public DateTimeOffset Fecha  { get; init; }
}
```

5.5 GET /health

GET **/health** *Estado de Integración para circuit breaker de Polly*

Propietario: P3 **Consumidor: P1** **Consumidor: P4** **Consumidor: Monitor**

```
public record HealthResponse
{
    // "Healthy" | "Degraded" | "Unhealthy"
    public string Status              { get; init; }
    public bool   CoreDisponible      { get; init; }
    public bool   MirrorActualizado   { get; init; }
    public int    ColasPendientes     { get; init; }
    public DateTimeOffset UltimaSync { get; init; }
    public Dictionary<string, string> Checks { get; init; }
    // "db"   → "Healthy"  |  "core" → "Degraded"  |  "nsb" → "Healthy"
}
```

Sección 06 · Eventos & Comandos NServiceBus

Convención: ICommand = flujo dirigido, un consumidor. IEvent = publicación broadcast, múltiples consumidores. El propietario es siempre la capa que publica el mensaje.

6.1 VentaRealizadaOfflineMessage [CMD]

CMD **VentaRealizadaOfflineMessage** *P1 → P3 · Encolar venta cuando Integración no está disponible*

Propietario: P1 Caja **Consumidor: P3 Integración** **CRITICO**

```
public record VentaRealizadaOfflineMessage : ICommand
{
    // Se convierte en Idempotency-Key al reenviar al Core
    public Guid     IdTransaccionLocal   { get; init; }
    public string   IdCajero             { get; init; }
    public string   IdSucursal           { get; init; }
    public Guid     ClienteId            { get; init; }
    public string   MetodoPago           { get; init; }
    public decimal  MontoTotal           { get; init; }
    public decimal  MontoRecibido        { get; init; }
    public IReadOnlyList<LineaVentaDto> Lineas { get; init; }
    public DateTimeOffset FechaLocal     { get; init; }
}
```

6.2 VentaSincronizadaEvent [EVT]

EVT **VentaSincronizadaEvent** *P3 → P1 · Confirmar que la venta offline fue aplicada en Core*

Propietario: P3 Integración **Consumidor: P1 Caja**

```
public record VentaSincronizadaEvent : IEvent
{
    public Guid   IdTransaccionLocal     { get; init; }
    public Guid   FacturaIdCore          { get; init; }
    public string NumeroFactura          { get; init; }
    // "Sincronizada" | "RechazadaCore"
    public string Resultado              { get; init; }
    public string? MotivoRechazo         { get; init; }
    public DateTimeOffset SincronizadaEn { get; init; }
}
```

6.3 InventarioActualizadoEvent [EVT]

EVT **InventarioActualizadoEvent** *P2 → P3 · Notificar cambios de stock para refrescar mirror*

Propietario: P2 Core **Consumidor: P3 Integración**

```
public record InventarioActualizadoEvent : IEvent
{
    public int      ProductoId       { get; init; }
    public int      StockNuevo       { get; init; }
    public int      StockAnterior    { get; init; }
    // "Venta" | "Ajuste" | "Compra" | "Devolucion"
    public string   Motivo           { get; init; }
    public DateTimeOffset Fecha      { get; init; }
}
```

6.4 AplicarTransaccionesOfflineCommand [CMD]

CMD **AplicarTransaccionesOfflineCommand** *P3 Saga → P2 · Reenvío lote tras reconexión*

Propietario: P3 Saga **Consumidor: P2 Core API** **CRITICO**

```
public record AplicarTransaccionesOfflineCommand : ICommand
{
    public Guid   SagaId         { get; init; }
    public IReadOnlyList<VentaOfflineDto> Transacciones { get; init; }
}

public record VentaOfflineDto
{
    // = IdTransaccionLocal → Idempotency-Key en Core
    public Guid   IdempotencyKey    { get; init; }
    public CrearVentaRequest Payload { get; init; }
    public DateTimeOffset FechaOriginal { get; init; }
}
```

6.5 OrdenCambioEstadoEvent [EVT]

EVT **OrdenCambioEstadoEvent** *P2 → P3 → P4 · Propagar cambios de estado al portal web*

Propietario: P2 Core **Consumidor: P3 → P4 Website**

```
public record OrdenCambioEstadoEvent : IEvent
{
    public Guid   OrdenId          { get; init; }
    public string EstadoNuevo      { get; init; }
    public string EstadoAnterior   { get; init; }
    public string? Nota            { get; init; }
    public DateTimeOffset Fecha    { get; init; }
    // P3 reenvía al Website vía SignalR o polling
}
```

6.6 ReconciliacionCompletadaEvent [EVT]

EVT **ReconciliacionCompletadaEvent** *P3 → P1 · Notificar sync completo a Caja*

Propietario: P3 Integración **Consumidor: P1 Caja**

```
public record ReconciliacionCompletadaEvent : IEvent
{
    public string SucursalId              { get; init; }
    public int    TotalTransacciones      { get; init; }
    public int    Aplicadas               { get; init; }
    public int    Rechazadas              { get; init; }
    public DateTimeOffset CompletadaEn    { get; init; }
}
```

Sección 07 · Modelos Internos del Core (P2)

7.1 FacturaDto — Documento fiscal completo

GET **FacturaDto** *Factura con ITBIS, líneas de detalle y clave de idempotencia*

Propietario: P2 Core **Consumidor: P3** **Consumidor: Reportes** **CRITICO**

```
public record FacturaDto
{
    public Guid    Id                    { get; init; }
    public string  NumeroFactura         { get; init; }  // Numeración secuencial
    public Guid    ClienteId             { get; init; }
    public string  NombreCliente         { get; init; }
    public string? RncCedulaCliente      { get; init; }
    public string  CajeroId              { get; init; }
    public string  SucursalId            { get; init; }
    public string  MetodoPago            { get; init; }
    public decimal Subtotal              { get; init; }
    public decimal Itbis                 { get; init; }  // 18%
    public decimal Total                 { get; init; }
    public decimal Cambio                { get; init; }
    public DateTimeOffset FechaHora      { get; init; }
    // "Confirmada" | "Anulada"
    public string  Estado                { get; init; }
    public IReadOnlyList<LineaFacturaDto> Lineas { get; init; }
    public Guid?   IdempotencyKey        { get; init; }
}

public record LineaFacturaDto
{
    public int     Cantidad              { get; init; }
    public string  Descripcion           { get; init; }
    public decimal PrecioUnitario        { get; init; }
    public decimal Descuento             { get; init; }
    public decimal Itbis                 { get; init; }
    public decimal Total                 { get; init; }
}
```

7.2 CotizacionDto — Flujo a factura

GET **CotizacionDto** *Cotización con estados y referencia a factura al convertirse*

Propietario: P2 Core **Consumidor: P3** **Consumidor: P1**

```
public record CotizacionDto
{
    public Guid    Id                  { get; init; }
    public string  NumeroCotizacion    { get; init; }
    public Guid    ClienteId           { get; init; }
    public string  VendedorId          { get; init; }
    public decimal Total               { get; init; }
    // "Borrador"|"Enviada"|"Aprobada"|"Rechazada"|"Facturada"
    public string  Estado              { get; init; }
    public DateTimeOffset Fecha        { get; init; }
    public DateTimeOffset? Vencimiento { get; init; }
    public Guid?   FacturaId           { get; init; }
    public IReadOnlyList<LineaFacturaDto> Lineas { get; init; }
}
```

7.3 AuditLogEntry — Tabla inmutable de auditoría

GET **AuditLogEntry** *INSERT-only. Sin UPDATE ni DELETE al rol de la app*

Propietario: P2 Core **Consumidor: Admin**

```
// INSERT-only. Sin UPDATE ni DELETE permitidos al rol de la app.
public record AuditLogEntry
{
    public long   Id              { get; init; }
    public string UserId          { get; init; }
    // "Core" | "Integracion" | "Caja" | "Website"
    public string Layer           { get; init; }
    public string Accion          { get; init; }
    public string Entidad         { get; init; }
    public string? EntidadId      { get; init; }
    public string? PayloadJson    { get; init; }  // sin passwords/tarjetas
    public string IpOrigen        { get; init; }
    public string CorrelationId   { get; init; }
    public int    HttpStatus      { get; init; }
    public DateTimeOffset Fecha   { get; init; }
}
```

7.4 *IntegrationLogEntry — Log de Integración*

GET **`IntegrationLogEntry`** *Registro de peticiones IN/OUT con latencia y flag de caché*

Propietario: P3 Integración **Consumidor: Monitor** **Consumidor: Admin**

```
public record IntegrationLogEntry
{
    public long   Id              { get; init; }
    public string Endpoint        { get; init; }
    // "IN" | "OUT"
    public string Direccion       { get; init; }
    public string? RequestJson    { get; init; }
    public string? ResponseJson   { get; init; }
    public int    HttpStatus      { get; init; }
    public int    LatenciaMs      { get; init; }
    public bool   DesdeCache      { get; init; }
    public string CorrelationId   { get; init; }
    public DateTimeOffset Fecha   { get; init; }
}
```

Sección 08 · Tabla Resumen de Endpoints REST

Todos los endpoints son expuestos por P3 (Integración) como API Gateway. P3 es el árbitro de breaking changes.

Método	Endpoint	Propietario	Consumidor	Descripción
POST	`/api/v1/auth/login`	P3 (proxy P2)	P1, P4	Autenticación JWT
POST	`/api/v1/auth/refresh`	P3	P1, P4	Renovar token
GET	`/api/v1/productos`	P3	P1, P4	Lista paginada (offline OK)
GET	`/api/v1/productos/{id}`	P3	P1, P4	Detalle con stock
GET	`/api/v1/servicios`	P3	P1, P4	Lista de servicios
GET	`/api/v1/clientes/{id}`	P3	P1	Datos cliente / anónimo
POST	`/api/v1/ventas`	P3 → P2	P1	Crear venta (Idempotency-Key)
POST	`/api/v1/caja/inicio-dia`	P3	P1	Apertura de caja
POST	`/api/v1/caja/movimiento`	P3	P1	Entrada / salida efectivo
POST	`/api/v1/caja/cierre-dia`	P3	P1	Cuadre y cierre
POST	`/api/v1/ordenes`	P3 → P2	P4	Orden web (async)
GET	`/api/v1/ordenes/{id}/estado`	P3	P4	Seguimiento orden
POST	`/api/v1/pagos/simular`	P3 (ext)	P4	Simular pago externo
GET	`/api/v1/cuentascobrar/{id}`	P3 → P2	P1	CxC con abonos
GET	`/health`	P3	P1, P4, Monitor	Estado del sistema

Sección 09 · Tabla Resumen de Mensajes NServiceBus

Tipo	Mensaje	Publicado por	Consumido por	Propósito
CMD	`VentaRealizadaOfflineMessage`	P1 Caja	P3 Integración	Encolar venta sin conexión
EVT	`VentaSincronizadaEvent`	P3	P1	Confirmar venta en Core
EVT	`InventarioActualizadoEvent`	P2 Core	P3	Refrescar mirror de stock
CMD	`AplicarTransaccionesOfflineCommand`	P3 Saga	P2 Core	Reenvío lote tras reconexion
EVT	`OrdenCambioEstadoEvent`	P2	P3 → P4	Propagar estado al portal
EVT	`ReconciliacionCompletadaEvent`	P3	P1	Notificar sync completo